Vision-Based Recognition and Actuation of Unconnected Home Controls

Geonhak Lee
Dept. Information System
Hanyang University
Seoul, Republic of Korea
thisiscrane@hanyang.ac.kr

Jemin An
Dept. Information System
Hanyang University
Seoul, Republic of Korea
ajm6396@hanyang.ac.kr

Jiwon Han
Dept. Information System
Hanyang University
Seoul, Republic of Korea
hjw9657@hanyang.ac.kr

Hyunseo Choi
Dept. Information System
Hanyang University
Seoul, Republic of Korea
hyunseo615@hanyang.ac.kr

***Abstract*—This study introduces a vision-based actuation system that lets a proactive smart-home agent operate the devices that are not on the network. Many homes still have wall switches, toggles and power strips that no routine can reach unless someone registers them by hand, and a command that is sent is usually assumed to have worked. We design the whole process, from finding a control in a camera image to running it inside a routine, as seven stages in three phases: setup, runtime and learning.**

Wireless control (IR, Wi-Fi, Bluetooth) comes first. When a computer-vision check shows that a wireless command failed, and a safety check has passed, a robot presses the control by hand. The robot only needs to receive a target position and an action, so the recognition of controls, the mapping between controls and devices, and the checking of results are the main work of the system.

By combining vision-based recognition with verified control and a risk-based autonomy policy, the system offers a way to bring unconnected controls into proactive routines without replacing them. This proposal gives the motivation, the client's needs, the proposed solution, related software, the detailed requirements, the development environment and the specifications.

***Index Terms*—Proactive Home, Smart Home, Service Robot, Vision-Language Model, Computer Vision, Closed-Loop Actuation**

Roles	Name	Task description and etc.
AI Developer	Geonhak Lee	An AI Developer builds the vision and language parts: control detection, the VLM that reads controls and infers intent, and the CV state check. The role also builds the robot motion planner and tunes recognition accuracy.
Backend Developer	Jemin An	A Backend Developer builds the Control Orchestrator, the IR, Wi-Fi and Bluetooth channel adapters and the Robot Abstraction Layer, and handles the safety gate and the learning updates. The role also writes tests.
Backend Developer	Jiwon Han	A Backend Developer builds the Mapping Registry, the Routine Engine, the database and the APIs for the app, and writes the execution logging. The role also integrates the cloud VLM service and writes tests.
Frontend Developer	Hyunseo Choi	A Frontend Developer builds the resident's app: the screens for reviewing routines and giving feedback, confirming mappings and receiving notifications, the setup guide, the plain-language execution log, and the switches that pause proactive features. The role connects the UI to the backend APIs.

I. Introduction

A. Motivation

Today's smart-home routines follow simple rules: when a time or a sensor condition is met, the home controls the devices that were registered. This works only if every control the resident cares about is on the network and has been registered. Many are not. Wall switches, toggles and power strips have no network connection, and in rented homes the wiring often cannot be changed. So a typical home is a mix: a few connected appliances next to many controls that only a hand can operate.

Two developments make a different approach realistic. First, low-cost robot arms, linear actuators and cameras are now widely available, and general-purpose home robots that can move around a home and reach a wall switch panel are being developed. Open-source desktop arms and small linear actuators are sold at a modest price, and a camera can show them where a control is. Second, vision-language models (VLMs), which turn images into language-based understanding, and VLA research such as RT-2 and OpenVLA show that a machine can understand and act on what it sees. Together, these suggest that a system could find the controls in a home, work out what each one operates, and have a robot press it when a routine needs it.

There is one problem. A robot that presses controls reaches only a limited area, and by itself it does not know which control to press, how to press it, or whether the press worked. A system that simply sends press commands would repeat the weakness of today's routines: the command is assumed to have worked. So we want a system that recognizes the controls, decides how each one is best operated, and checks the result.

B. Problem Statement (Client's Needs)

The client is the resident of a home that has both connected devices and unconnected controls (old switches and buttons with no network connection). Four problems stand between this resident and a home that acts before being asked. First, a routine cannot target a control that has no network connection, so the most common household controls are left out of automation. Second, the resident has to register every device and its control method by hand, so automation grows only as

fast as the resident's patience. Third, a command that is sent is assumed to have worked. If a wireless command fails, nobody finds out until the resident notices, and there is no second try. Fourth, a robot that presses controls in a home must be safe, because an action that is unsafe or unwanted is worse than no action.

From these problems we list what the client needs. The resident needs a system that (N1) works with unconnected controls without replacing them; (N2) needs little manual setup; (N3) checks that an action worked and recovers when it did not; (N4) stays safe, private and under the resident's control; (N5) works with simple linear actuators now and with other robots later; and (N6) acts before being asked and improves with use.

C. Proposed Solution

We propose a system that uses a camera to find controls, works out what each control operates, adds the confirmed controls to routines, and later runs those routines with a fallback when a command fails. The system follows one end-to-end process of seven stages (S1 to S7) in three phases. For every stage, we define what goes in, what comes out, and what happens on failure. Wireless control comes first. If a computer-vision (CV) check shows that it failed, a robot presses the control by hand, but only after a safety check.

What we assume about the robot. We assume a general-purpose home robot, with a camera head, arms and a mobile base (wheels, tracks or legs), that can move to a control, see both the control and its own hand, and press it. The robot receives a target position and an action, such as press or flip, and does nothing more. It is hidden behind a Robot Abstraction Layer (RAL), so a different robot, a simulator or a simple script can be swapped in. In the prototype, we stand in for such a robot with linear actuators that push straight, carried on a wheeled cart that is positioned by hand (Section III). The robot is kept simple on purpose: finding the control, deciding how to operate it and checking the result are done by the system.

Phase 1, setup (S1 to S3). In S1, the system finds and registers the controls in the home. An object detector that works from text descriptions looks for wall switches, toggles and power strips in the robot's camera feed. A VLM reads the type and state of each one, and depth data, if the camera provides it, helps place it. Each control is saved as a record, together with a picture fingerprint (an embedding), so that it can be recognized again later. In S2, the system links each control to a device. It can watch as the resident uses the control, or press it and compare the device before and after by camera. A link is confirmed only when the change is clear in the picture and the VLM agrees. For each device, the system also sends a test command through each wireless channel (IR, Wi-Fi, Bluetooth) and keeps only the channels that work, so a device can be reached both wirelessly and through the control that the actuator can press. In S3, confirmed links are used to build routines. The system creates one from repeated habits and registers it automatically, and the resident can also build one by hand. The resident is told right away and can give feedback at any time: keep the routine, change it or remove it.

Phase 2, runtime (S4 to S6). In S4, a sensor, a time or a routine starts a run, and a cloud VLM works out what the resident wants (the intent) and which device is the target. In S5, the system looks up the saved link, ranks the device's wireless channels by a score learned from past results, sends the command through the best one, and checks the result. The main check is whether the device itself changed, which the camera can see. As a second check, the system looks for changes around the device, such as the room getting brighter. If every wireless channel fails, or the device has none, S6 begins. CV finds the control, a motion planner (a VLA model or a simple script) plans the movement, a safety check looks for people and obstacles, and only then does the robot press the control. If the safety check fails, the action is cancelled and the resident is told. Whether the device was operated wirelessly or by the robot, the checked result is written to a log the resident can read, and the resident can choose whether to be notified of successful runs as well as failed ones.

Phase 3, learning (S7). Every run, even a cancelled one, teaches the system. It updates how often each channel works, how sure it is about each link, and how useful each routine is. In the next run, the earlier phases use these values. The camera feed from inside a home is private, so frames are processed at the edge (inside the home), only small crops of controls and short notes go to the cloud, and proactive features stay off until the resident turns them on.

D. Research on Related Software

Several existing platforms and research systems cover a part of this problem. We describe each one and say what it leaves open.

1) **LG ThinQ and ThinQ ON**
   LG ThinQ is LG's AI home platform, and ThinQ ON is its home hub. Together they let LG's appliances work as one system. In 2024, LG bought 80% of Athom, the maker of Homey, which shows that LG wants to bring third-party devices into its ecosystem as well. ThinQ is strong in scale: it has many connected appliances and a natural source of requests. However, as far as we know, its public descriptions do not cover operating unconnected switches, or checking by camera that a command produced the intended result. Our system is meant to sit under such a platform and add these two things.
2) **Homey**
   Homey, made by Athom, is a smart-home hub. Its Pro model supports Wi-Fi, Bluetooth Low Energy, Matter, Thread, Zigbee, Z-Wave and infrared, and it runs locally. Users build automations in Flow, a rule editor, and use Advanced Flow and HomeyScript for more complex logic. Its support for many connection types is a good model for our Control Orchestrator, which also hides different channels behind one interface. However, the user must write the rules, an IR command returns no confirmation

that the device responded, and a plain wall switch needs an extra product such as a button pusher. We add camera-based checking and an automatic path to pressing controls by hand.

3) **SwitchBot Bot**
SwitchBot Bot is a small button pusher. It is stuck next to a rocker switch or a one-way button with 3M adhesive, and it presses the button when the SwitchBot app asks it to, over Bluetooth. With the SwitchBot Hub, it can also be used remotely or by voice. It is an inexpensive way to bring old switches into a smart home without rewiring, which suits renters. Its limits come from its design: one unit must be bought and attached for every switch, each unit carries its own battery, and its documents do not describe checking that the pressed device really turned on. Our system differs in two ways: one camera-guided robot can serve several switches, and the result of every press is checked.

4) **SwitchBot AI Hub**
SwitchBot AI Hub is one of the closest products we found. It is marketed as a home AI agent that uses connected cameras as its eyes and smart devices as its hands. It uses a VLM to understand what happens in the home and manages routine tasks and security on its own. It runs as a local hub, includes Home Assistant, and connects SwitchBot devices to Matter, Apple Home and Google Home. Its focus is understanding the scene and automating around it. The product page does not describe checking whether a single device command really worked, or switching to a physical press when it did not. We address this narrow but important last step, and our system could in principle sit under such an agent.

5) **Robot Arms**
Several low-cost robot arms are sold for education and research. SO-101, an open-source arm from Hugging Face, is built from six servo motors and works with the LeRobot software. Its base kit is listed at about 100 US dollars, and assembled units can cost several times more. DOBOT Magician is a desktop arm with four axes, a payload of 500 g and a reach of 320 mm. At the other end, assistive arms such as Kinova Jaco are made for people with limited arm movement and are priced by quotation. These arms are good at moving to a given position, and a camera can supply that position. What they do not include is any understanding of the home: which control is where, which device it operates, and whether the press worked. A fixed arm also reaches only a limited area, so in our design the arm is carried on a mobile base that brings it to each control. The arm is a replaceable hand behind the Robot Abstraction Layer, and our system supplies the recognition, mapping and checking. For the prototype we use an even simpler hand: linear actuators on the cart (Section III).

6) **Vision-Language-Action Models**
VLA models take camera images and language instructions and produce robot actions. RT-2 trains a VLM on web data together with robot movement data, and writes actions as text tokens. OpenVLA is an open-source model with 7 billion parameters, trained on 970,000 real robot demonstrations. π_0 adds a flow-matching action generator to a pretrained VLM and shows careful tasks such as folding laundry. These models give the robot the skill to press a control, but they do not know which device is behind it, whether pressing is the right way to control it, or whether the press worked. We treat the VLA planner as a part that can be swapped, pass it the target and its position, and check the result afterward.

7) **Open-Vocabulary Detection**
OWL-ViT and Grounding DINO find object categories in an image from a text query, without retraining. Wall switches, toggles and power strips come in many shapes and are not covered well by detectors with fixed categories, so a text query such as “wall switch” or “power strip” is a practical way to get candidates in S1. These detectors only locate objects. They do not say what a control operates, which is the job of our mapping stage.

8) **Closed-Loop Embodied Agents**
SayCan limits a language model to actions that the robot can really do in the current scene. Inner Monologue feeds back whether an action worked, and shows that this feedback helps a robot finish its instructions. Du *et al.* use a VLM to answer questions about an image and decide whether a task succeeded. These works show that visual feedback can check results, but they check whether a single task given to the robot succeeded. They do not manage a mixed set of household devices and channels, and they do not learn which channel to trust for which device.

As far as we know, none of these systems combines recognition of unconnected controls, device mapping, verified wireless control, a safety-gated physical fallback and learning. Platforms reach many devices but treat a sent command as done. Pushers and robots can press things but do not know which control needs which method. Closed-loop agents check whether a single task given to the robot succeeded rather than a mixed set of household devices. Our system is meant to be the layer that connects these pieces and works with different robots.

II. Requirements

This section lists what the system must do. The *resident* gives feedback on routines and has the final say. The *robot* moves to a control and presses it when needed. The *cloud VLM* works out the intent. The *smart-home hub* sends IR, Wi-Fi and Bluetooth commands.

1) **Control Recognition**
From the robot’s camera feed, the system finds wall switches, toggles and power strips and decides the type of each one. For every control it saves a record with the type, the visible state, the possible actions, its position and a picture fingerprint (an embedding), so the same control is recognized later and no duplicate is created. A control

that cannot be told clearly is skipped until a clearer frame is available.

2) **Device Mapping**
The system suggests which device each control operates. It can watch: when the resident uses a control and a nearby device visibly changes (a switch is flipped and a fan starts to turn), that pair becomes a candidate. It can also test: the robot presses the control and the system compares the device before and after by camera. Each candidate is saved as Pending with a confidence value. A mapping becomes Verified only when the observed change is clear and the VLM agrees that it belongs to the mapped device. The resident can reject a mapping at any time. If a control looks different or has moved, its mapping becomes Stale and is verified again before use.

3) **Control-Method Registration**
For each device, the system registers the control methods that might work: IR, Wi-Fi, Bluetooth, or a physical press by the actuator. It sends a test command by each wireless method and keeps a method only if state verification confirms the expected change. A device with no working wireless method is saved as a target for the actuator. If registration fails, the system retries a few times and then asks the resident to choose a method or check the device by hand.

4) **Routine Registration and Feedback**
A verified mapping can be used in a routine, and such a routine can be registered without waiting for approval. When the system notices a repeated habit, such as switching on the living-room light at about the same time every day, it registers a routine and tells the resident what was added. The resident can also build a routine by hand from any verified control. After that, the resident can keep it, change it, remove it, or change how independently it acts. Routines the resident often rejects get lower priority, and accepted routines get higher priority.

5) **Trigger and Intent**
The system accepts sensor events, time events and routine events as triggers and attaches the current situation to each one: the time, the room and the state of nearby devices. Duplicate triggers within a short time are merged, and each trigger starts one run with its own identifier. The Routine Engine sends the situation and a camera image, limited to crops of the relevant controls, to the cloud VLM, which returns the intent, the target device and a confidence value. If the cloud cannot be reached, the system does not act on its own and only suggests.

6) **Autonomy by Risk**
What the system does depends on how risky the device is, how confident it is, and whether the action turns the device on or off. For low-risk devices such as lights, fans, a TV or an air purifier, it acts and then notifies the resident when confidence is high, asks first when it is medium, and only suggests when it is low. High-risk devices such as heaters, electric blankets, microwaves and power strips that feed them can also be added to routines, but the resident sees a warning when the routine is registered, and every run sends a clear notification. Turning such a device off follows the same rules as a low-risk device. Turning it on always needs the resident's approval first, and the system does nothing if there is no answer. If the resident keeps rejecting a routine, its independence goes down one step.

7) **Channel Selection and Wireless Command**
Before sending a command, the Control Orchestrator looks up the target in the Mapping Registry. Without a verified mapping the system does not act, or, with the resident's consent, first maps that device. The Orchestrator ranks the device's wireless channels, preferring ones that have worked often and are fast, sends the command through the best one and, if state verification fails, tries the next. The physical press is used only when the device has no wireless channel or every wireless channel has failed.

8) **State Verification**
After each command, the State Verifier checks whether the device really changed. The main check looks at the device itself: unless the device is already in view, the robot goes to the spot saved with the mapping, and the camera takes a picture of the target and compares it with what should have happened, such as a lit lamp, a spinning fan or a lit display. The second check looks at the surroundings, for example the room getting brighter. The main check decides the result; the second check supports it, and any disagreement is written to the log. The result is success, failure or unsure. Success ends the run. Failure moves on to the next channel or, when no wireless channel is left, to the actuator. An unsure result is checked once more after a short wait, without operating the device again. If it is still unsure, the run stops and the result is shown to the resident with a picture of the device, and the resident chooses whether to try again or leave it as it is.

9) **Robot Motion Planning**
When a physical press is needed, the robot first moves to the control's saved press spot, where the control is visible and within reach of an actuator, and the system then finds the control again in the current camera image, starting from its saved position and fingerprint. If it cannot be found or its position is not reliable enough, the run is cancelled and the resident is told. Otherwise the planner receives the control, its position and the expected effect, and returns which actuator to use and how long to push. Linear actuators have no joints, so there is no path to plan. The planner is a simple script, used through the Robot Abstraction Layer so that a VLA model or a different robot can replace it later.

10) **Safety Gate**
Before any movement, the Safety Guard checks that no person or pet is near the robot's route or the actuators' work area, that the action is allowed by the risk policy (Requirement 6), and that the position of the control is reliable enough. If any check fails, the action is cancelled

and the resident is told why. An emergency stop is always available and halts any movement at once.

11) **Physical Execution and Result Check**
When the safety gate passes, the robot presses the control through the Manipulator Adapter, and state verification runs again. If the check reports a failure, the system may try again a limited number of times, running the safety gate before every try, and then cancels and tells the resident. An unsure result is handled as in Requirement 8 and never leads to another press.

12) **Learning and Log**
After each run, the system updates how often the channel it used has worked and how fast it was, giving more weight to recent results, so the next ranking improves. After a successful press, the spot where the robot stood is saved as the control's press spot, so the robot goes there first next time. A mapping loses confidence when a later check disagrees with it, gains it when the check agrees, and becomes Stale if the confidence gets too low or the resident rejects the result. Every run, including a cancelled one, writes a log entry with the trigger, intent, confidence, channel, the result of each stage, the main and second checks and the reason for any cancellation. The resident can read the intent, channel and reason of any automatic action in plain language.

III. Development Environment

A. Choice of Software Development Platform

1 Operating System (OS)

1) **Windows**
Windows is Microsoft's desktop operating system. It runs Python, Node.js and Unity and works well with USB devices such as cameras. Some members develop on Windows. Our reference machine for speed tests is a Windows 11 laptop with 16 GB of RAM.

2) **macOS**
macOS is Apple's operating system, with a Unix-based command line and the Homebrew package manager. Some members develop on macOS. Because the team uses both systems, we keep the code cross-platform and test the hardware drivers (camera and actuator controller) on both.

2 Programming Languages

1) **Python 3.11**
Python is the main language of the project. It is used for the edge hub (camera, channel adapters and actuator control), the AI parts (detector, VLM calls, state checks) and the backend. We choose it because PyTorch, OpenCV and the Ultralytics YOLO package are all Python libraries.

2) **TypeScript 5**
TypeScript is JavaScript with types, which catches many errors before the program runs. We use it with React to write the resident's web app: the screens for confirming mappings, giving feedback on routines, reading the log and pressing the emergency stop. The app runs in any browser, so the resident installs nothing.

3) **Unity 6 LTS (C#)**
Unity is a real-time 3D engine. We use it only as an auxiliary tool: a simulator of the switch panel and the robot, used for the rough demo before the real demo and for tests when the real robot is not available. It connects to the Python hub through a WebSocket. It is not the main result of the project.

3 Cloud Infrastructure

1) **Amazon EC2 (AWS)**
Amazon Web Services (AWS) is Amazon's cloud platform, where computing power is rented and paid for by use. EC2 gives us virtual machines. One small machine runs the backend, which holds the Mapping Registry, the routines and the log and makes the VLM calls. A GPU machine is rented only for the hours we need to tune the detector.

4 Development Tools

1) **Visual Studio Code**
Visual Studio Code is a light source-code editor from Microsoft. It is our main editor for Python and TypeScript, with extensions for debugging, formatting and Git.

2) **GitHub**
GitHub is a cloud platform for version control and teamwork, built on Git. We keep the code in public repositories, and the photo dataset and the trained detector in a private repository. We use branches, pull requests, issues and code review. The LaTeX files of our documents are stored there as well.

3) **Overleaf**
Overleaf is an online LaTeX editor. We write the proposal and later documents together in it, starting from the IEEE conference template.

4) **Claude Opus 5.5 (AI tool)**
Claude Opus 5.5 is an AI model from Anthropic. We use it during development to draft and review code, explain error messages and check our documents. The members decide and test everything, and Claude is not a part of the product. The cloud VLM of the product is chosen separately after testing.

5 Cost Estimation

Table I lists the estimated cost of the prototype. Each of the four members counts one laptop of 2,000,000 KRW. The software tools are free or open source. The prices of the actuator, the controller parts and the webcam are those in our shopping cart before purchase (November 2026), and the shipping cost of that order is included.

6 Versions and Computing Resources

- Main versions: Python 3.11, PyTorch 2, Ultralytics YOLO, OpenCV 4, FastAPI, Node.js 20, TypeScript 5, React 18 and Unity 6 LTS, pinned in the repository.
- Fine-tuning of the detector runs on a rented cloud GPU. We expect the small detector to run on a laptop CPU, since the scene is mostly still, and we will measure this.

TABLE I
ESTIMATED PROTOTYPE COST (KRW)

Item	Purpose	Cost
Laptops (4 members, 2,000,000 each)	development and tests	8,000,000
Linear actuator (12 V, 100 mm)	physical press (S6)	37,000
BTS7960 motor driver module	drives the actuator	5,300
Arduino Nano (USB-C)	controller board, takes commands over USB	3,490
12 V 2 A power adapter and DC jack	powers the actuator	16,210
Jumper wires (40 pin)	wiring	1,500
USB webcam (C270)	camera feed	23,040
Shipping	for the parts above	8,700
Software	VS Code, GitHub, Overleaf, Unity (free plans)	0
Total		8,095,240

B. Software in Use

We reuse existing software and build only the parts that connect it.

1) **YOLO (Ultralytics)**
   A fast object detector. We fine-tune it on our own photos of switch panels to find wall switches, toggles and power strips.
2) **Grounding DINO and OWL-ViT**
   Open-vocabulary detectors that find objects from a text query such as "wall switch", without retraining. We use them to draft labels and to find controls before any training.
3) **Cloud VLM (through an API)**
   An existing vision-language model that reads the type and state of a control, infers the resident's intent and answers yes-or-no questions when we check a result. We do not train it.
4) **DINOv2**
   A pretrained image encoder. Its embedding of a control's picture is the fingerprint that lets the system recognize the same control again.
5) **OpenCV**
   A computer-vision library. We use it for camera capture and for handling images, such as cropping a control and comparing two pictures.
6) **Linear actuators and a driver board**
   A linear actuator is a small electric motor that pushes a rod straight out and back, which is exactly the motion of pressing a button. We use 12 V, 100 mm actuators, mounted side by side on the cart. A BTS7960 motor driver module sends power to an actuator in either direction (out or back), and an Arduino Nano controller board receives simple commands (push, release) from the Python hub over a USB cable and tells the driver what to do. A 12 V, 2 A power adapter supplies the power. The cart is placed so that one of the actuators faces the control. Without joints there is nothing to plan: the system only chooses which actuator to use and how long to push.
7) **VLA models (RT-2, OpenVLA, π_0)**
   Models that turn camera images and instructions into robot actions. They are candidates to replace the simple script as the motion planner later.
8) **FastAPI and SQLite**
   FastAPI is a Python web framework for the backend, and SQLite is a small database file. Together they hold the Mapping Registry, the routines and the log in the prototype.
9) **React**
   A JavaScript library for building user interfaces. It is used with TypeScript for the web app.

Related commercial products (LG ThinQ ON, Homey, SwitchBot Bot, SwitchBot AI Hub) are compared in the Introduction.

C. Task Distribution

The roles are those in the table on the first page. By requirement:

- AI developer (Geonhak Lee): Requirements 1, 8 and 9, which are recognition, visual checking and the robot motion planner.
- Jemin An: Requirements 3, 6, 7, 10 and 11, which are channels, autonomy policy, command sending, safety and physical execution.
- Jiwon Han: Requirements 2, 4, 5 and the log in Requirement 12, which are mapping, routines, triggers and logging.
- Hyunseo Choi: the app screens for confirming mappings, giving feedback on routines and reading the log.
- The learning updates in Requirement 12 are shared by the two backend developers. We will refine all of this in the design phase.

IV. SPECIFICATIONS

This section explains in plain words how each requirement will be built. The numbers in the lists are first proposals that we tune after the first measurements.

A. What the System Remembers and How the Parts Talk

- Controls: for each control, its type, how it looks now, what can be done with it, where it is on the panel, a picture "fingerprint", and the spot where the robot stands to press it.
- Devices: name, type, room, how risky the device is, and which channels can operate it.
- Mappings: which control operates which device, the spot from which the robot saw the device change when the link was verified, a confidence score and a status: Pending (waiting for proof), Verified (confirmed), Rejected (by the resident) or Stale (must be checked again).
- Channels: for each device and channel (IR, Wi-Fi, Bluetooth or the actuator), how often it works and how fast it is.
- Routines and runs: a routine keeps the habit, its priority and how independently it acts. A run is one action from start to finish and keeps what triggered it, the intent, the channel used, the result and, if it was cancelled, the reason.

- The web app asks the backend for the lists of controls, mappings and routines, and sends the resident's choices: confirm or reject a mapping, feedback on a routine, and the emergency stop. The backend pushes live updates to the app, such as a new control, a new routine or the progress of a run.
- The robot is used through one common interface (go to a spot, press, release, stop), so the linear actuators on the cart, the Unity simulator or a simple script can be swapped in. In the prototype, "go to a spot" asks a team member to push the cart there. Each wireless channel is a small adapter with the same two abilities: tell whether the device can be reached, and send a command.

B. Control Recognition (Requirement 1)

The edge hub looks at the robot's camera feed, finds each control and remembers it so that it can be recognized later. **How it works:**

- A small YOLO detector, fine-tuned on our own photos, looks for three kinds of control: wall switch, toggle and power strip. It checks every fifth frame to save computing power.
- Our photos show real wall switches, toggles and power strips in different places, from different angles and in different light. A tool drafts the boxes first and we correct them by hand.
- We take the photos in sessions. One session is one visit to one place at one time of day, with its own light and angles. The photos are split 8 to 2: 80% is used to revise the prompts and the labels, and 20% is kept only for evaluation and never used for revision.
- The evaluation 20% is not taken from a single session. Every session gives 20% of its own photos, so the evaluation set has the same mix of places and light as all the data. Photos taken within a few seconds of each other go to the same side, so an almost identical picture is never in both. The 20% is used only to measure the result, for example how many controls are found with the right type and state.
- Each control found is cropped and shown to the VLM, which tells its type, its state (for example on or off) and whether the picture is clear enough.
- A picture fingerprint of the crop is compared with the saved ones. If it is very similar to a saved control, it is the same control and is not added twice.

C. Device Mapping (Requirement 2)

The system finds out which device each control operates, by watching the resident or by trying it, and accepts a link only when the evidence is clear. **How it works:**

- Watching: after the resident uses a control, the system compares the camera pictures of nearby devices for about 3 seconds.
- Trying: the robot presses the control and the system compares the device before and after (see State Verification).
- Candidates are ranked by simple hints: the same room, a short distance in the picture, and whether the VLM thinks the change fits the device. Each candidate starts as Pending.
- A link becomes Verified only when the change is clear and the VLM agrees. The resident can reject a link at any time. If the control looks different or has moved, the link becomes Stale and is checked again.
- When a link becomes Verified, the spot from which the robot saw the change is saved with it.

D. Control-Method Registration (Requirement 3)

For each device, the system finds out which ways of control really work. **How it works:**

- IR goes through a Wi-Fi IR blaster, Wi-Fi through a smart plug or bulb, and Bluetooth through a Python Bluetooth library.
- The system sends one test command per method, waits up to about 5 seconds and keeps the method only if the result check says it worked.
- A failed test is retried up to 3 times. After that, the app asks the resident to pick a method or check the device by hand.
- If no wireless method works, the actuator is the only way, and the device is marked as an actuator target.

E. Routine Registration and Feedback (Requirement 4)

Repeated habits become routines on their own, and the resident can change them at any time. **How it works:**

- A habit is the same device and action at about the same time of day (within a 30-minute window) on at least 3 days.
- A routine remembers the link, the time, a priority and how independently it acts: do it and tell the resident, ask first, or only suggest.
- When a routine is added, the app shows it, and the resident can keep, change or remove it.
- If the resident rejects a routine three times in a row, it becomes one step less independent.

F. Trigger and Intent (Requirement 5)

Events start runs, and a cloud VLM works out what the resident wants. **How it works:**

- Events come from sensors, a clock and routines. The same event arriving twice within 10 seconds is counted once.
- Each run gets its own ID and a short description of the situation: the time, the room and the state of nearby devices.
- The VLM receives that description and up to 3 small crops, and answers with the intent, the target device and a confidence value. It has about 8 seconds to answer.
- The final confidence mixes three things: the VLM's own confidence, how reliable the trigger is, and how often the resident accepted this routine before.
- If the cloud is slow or cannot be reached, the system only makes a suggestion.

G. Autonomy by Risk (Requirement 6)

A fixed policy decides whether to act, ask or only suggest. **How it works:**

- Devices are sorted into low risk (lights, fans, TV, air purifier) and high risk (heaters, electric blankets, microwaves and the

power strips that feed them). The sorting is kept in a settings file, so it can be changed without touching the program.
- There are two confidence levels, a high one (0.8) and a low one (0.5) as starting values.
- Low risk: act and tell the resident when confidence is high, ask first when it is medium, otherwise only suggest.
- High risk: a warning is shown when the routine is registered, and every run sends a notification. Turning the device off follows the low-risk rules. Turning it on always asks first and does nothing without an answer.

H. Channel Selection and Wireless Command (Requirement 7)

The Control Orchestrator looks up the link, ranks the device's wireless channels and tries them in order. **How it works:**
- It first looks up the link. Without a confirmed link the system does not act, unless the resident agrees to map the device first.
- Channels that often worked and are fast come first.
- Each channel gets up to 5 seconds, and the result is checked. If a channel fails, the next one is tried.
- If all channels fail, or the device has none, the run goes on to the actuator.

I. State Verification (Requirement 8)

A main check on the device and a second check on its surroundings decide success, failure or unsure. **How it works:**
- Main check: after the command, the robot goes to the spot saved with the mapping, unless the device is already in view, and the camera crops the device and the VLM answers a yes-or-no question about what should have happened (for example "is the lamp on?"). The picture is also compared with the one before.
- Second check: whether the room became brighter.
- The system waits up to 3 seconds for the effect to appear, for example for a fan to start.
- The second check decides only when the main check is unsure, and any disagreement is written to the log. If the result is still unsure, the system waits about 3 more seconds and checks once more, without operating the device again. If it is still unsure, the run stops, and the resident gets a picture of the device and chooses whether to try again or leave it as it is.

J. Robot Motion Planning (Requirement 9)

The robot goes to the control, finds it again in the current picture and turns it into something the actuator can act on. **How it works:**
- The robot goes to the control's saved press spot.
- The control is searched near its saved place by comparing fingerprints. If it is not similar enough (0.7 as a starting value), the run is cancelled and the resident is told.
- How deep to press is set for each kind of control.
- The planner is a swappable part. A simple script is used, and a VLA model could replace it later.
- The cart is placed so that one actuator faces the control, and the system only chooses which actuator to use. There is no path to plan.

K. Safety Gate (Requirement 10)

Every movement passes a gate, and an emergency stop is always available. **How it works:**
- A camera model looks for people and pets along the robot's route and in the work area of the actuators, before and during the movement.
- The linear actuators have a short stroke and a limited force. They press high-risk devices only when the risk policy (Requirement 6) allows it.
- The emergency stop can be started from a button in the app or from a key. It turns off the motors of the robot at once. The power adapter can also be switched off by hand.

L. Physical Execution and Result Check (Requirement 11)

The robot presses the control with an actuator, and the result is checked again. **How it works:**
- The actuator facing the control pushes out, holds briefly and pulls back.
- The linear actuators, the Unity simulator (for the rough demo and tests) or a simple script can be used, chosen in a settings file.
- There are at most 2 tries, made only after a failed check; an unsure result never leads to another press. The safety gate runs before each try, and the state check runs after each.
- If it still fails, the run is cancelled and the resident is told.

M. Learning and Log (Requirement 12)

Every run updates the channel scores and the link confidence and leaves a log entry. **How it works:**
- For each channel, the system keeps a running average of how often it works and how fast it is, and the newest result counts most (30 percent).
- A link's confidence goes up a little (0.05) when a later camera check agrees and down when it disagrees. If it falls below 0.2, or the resident rejects a result, the link becomes Stale.
- After a successful press, the spot where the robot stood becomes the control's new press spot.
- Every run, even a cancelled one, is written to the log with what triggered it, what was intended, which channel was used and the result.
- Explanations for the resident use fixed sentence templates, for example: "I switched on the fan at 21:05 by IR because of your evening routine. Checked: it worked."